**UNIVERSIDAD ADVENTISTA DEL PLATA**
**FACULTAD DE CIENCIAS ECONÓMICAS Y DE LA ADMINISTRACIÓN**
**ALGORITMOS Y ESTRUCTURA DE DATOS**

# LISTA DE EJERCICIOS Nº 8 — ALGORITMOS DE ORDENAMIENTO I: BUBBLE SORT E INSERTION SORT (U7)

## Sección 1: Pensar antes de programar

1. Un videojuego tiene una lista de jugadores con sus respectivos puntajes y se necesita mostrar solamente los 3 jugadores con mayor puntaje: *

```
[850, 1200, 430, 950, 1700, 620, 1100,
780]
```

   a. Una propuesta es: "Ordenar todos los jugadores de mayor a menor y luego tomar los tres primeros." ¿Es una solución válida? ¿Qué ventajas tiene?
   b. Piensa una alternativa que permita obtener los tres mayores sin ordenar necesariamente toda la lista. No es necesario programarla. Escribe los pasos en lenguaje natural.
      i. ¿Qué información tendrías que mantener durante el proceso para poder resolverlo?
      ii. ¿En qué situación podría convenir ordenar toda la lista de todas maneras?

## Sección 2: ¿Cómo se comporta un algoritmo de ordenamiento?

Tenemos:

```
[20, 11, 16, 9, 12, 14, 17, 19, 13, 15]
```

2. Bubble Sort: Sin ejecutar ningún programa:
   a. Aplica las dos primeras iteraciones de Bubble Sort (Ordenamiento Burbuja).
   b. Escribe cómo queda el array después de cada recorrido.
   c. Indica qué elementos fueron comparados.
   d. Indica qué intercambios se realizaron.
3. Insertion Sort: Realiza el mismo análisis utilizando Insertion Sort:
   a. Ejecuta las dos primeras iteraciones.
   b. Muestra cómo queda el arreglo después de cada una.
   c. Indica qué elemento se está insertando en cada iteración.
4. ¿Por qué los dos algoritmos de las preguntas 2 y 3 llegan al mismo resultado, pero recorren el problema de una manera diferente?

**Seccion 3. Tabla de posiciones de un torneo de Fútbol**

5. Crear una clase que represente un equipo de fútbol
    a. Que el equipo tenga sus puntos obtenidos en el torneo, los goles marcados y los goles en contra.
    b. En la clase (Equipo), agregar una función para calcular la diferencia de goles a partir de los goles marcados y los goles en contra.
    c. Crear una lista de al menos 5 equipos. Por ejemplo:

```
Boca           12 pts    GF 10    GC 4
River          12 pts    GF 8     GC 3
Racing         10 pts    GF 7     GC 6
Independiente  12 pts    GF 10    GC 5
San Lorenzo     9 pts    GF 6     GC 6
```

6. Utilizando Bubble Sort:
    a. Implementar un método que ordene los equipos según:
        i. mayor cantidad de puntos;
        ii. en caso de empate, mayor diferencia de goles;
        iii. en caso de nuevo empate, mayor cantidad de goles a favor.
    b. Mostrar la tabla antes de ordenar y después de ordenar.
7. Utilizando Insertion Sort:
    a. Implementar los mismos criterios de ordenamiento que el ejercicio anterior, pero utilizando Insertion Sort.
    b. Mostrar la tabla antes de ordenar y después de ordenar.
8. ¿Qué cambia entre Bubble Sort e Insertion Sort?
    a. Estrategia utilizada;
    b. Compara aproximadamente la cantidad de comparaciones y movimientos realizados por ambos algoritmos.
    c. Complejidad temporal.

**Sección 4: Eficiencia**

9. Considerá la siguiente tabla simplificada:

| Algoritmo | Mejor caso | Peor caso | Memoria auxiliar |
|---|---|---|---|
| Bubble Sort | $O(n^2)$ | $O(n^2)$ | O(1) |
| Insertion Sort | O(n) | $O(n^2)$ | O(1) |

    a. ¿Qué significa que un algoritmo tenga complejidad O(n)?
    b. ¿Qué significa que otro tenga complejidad $O(n^2)$?
    c. ¿Por qué Insertion Sort puede ser conveniente cuando los datos ya están casi ordenados?
    d. Ambos algoritmos utilizan O(1) de memoria auxiliar. ¿Qué significa esto?
    e. Supongamos dos situaciones:
        i. Situación 1: 20 elementos, memoria muy limitada.
        ii. Situación 2: 1.000.000 de elementos, mucho poder de procesamiento disponible.

iii. ¿Cambiaría tu elección de algoritmo? Explica qué factores tendrías en cuenta.

10. Tenemos tres listas:

```
A → 10 elementos
B → 1.000 elementos
C → 1.000.000 elementos
```

Y tres algoritmos:

```
Algoritmo 1 → O(n)
Algoritmo 2 → O(n log n)
Algoritmo 3 → O(n²)
```

a. Ordena los algoritmos de acuerdo con su crecimiento esperado cuando n aumenta.
b. ¿En cuál de las tres listas sería más preocupante utilizar un algoritmo $O(n^2)$?
c. ¿Significa esto que un algoritmo O(n) siempre será más rápido en la práctica? Justifica.
d. ¿Qué otras cosas, además de la complejidad temporal, pueden influir en la elección de un algoritmo?